

DATABASE_AND_API.md

YourBuy — Database & API Master Document

DATABASE ENGINE (VERY IMPORTANT)

The entire YourBuy project MUST use:

MySQL ONLY

Ignore all PostgreSQL references from older documentation.

All backend systems MUST remain:

- MySQL-compatible
- Prisma-compatible
- scalable
- production-oriented

Mandatory Prisma datasource:

```
datasource db {  
  provider = "mysql"  
  url      = env("DATABASE_URL")  
}
```

Environment variable format:

```
DATABASE_URL="mysql://USER:PASSWORD@HOST:PORT/DATABASE"
```

NEVER generate:

- PostgreSQL syntax
 - PostgreSQL-only migrations
 - PostgreSQL extensions
 - PostgreSQL infrastructure assumptions
-

1. BACKEND OVERVIEW

Backend Stack

- NestJS
 - TypeScript
 - Prisma ORM
 - MySQL
 - Redis
 - Socket.IO
 - JWT Authentication
-

2. BACKEND PHILOSOPHY

Backend architecture MUST:

- remain modular
 - remain scalable
 - remain maintainable
 - remain provider-agnostic
 - remain production-grade
-

Highest Priorities

1. Backend correctness
 2. Security
 3. Scalability
 4. Realtime stability
 5. Maintainability
-

3. ARCHITECTURE STYLE

Selected Architecture

Modular Monolith

Why

Provides:

- clean scaling
 - easier debugging
 - better maintainability
 - future microservice readiness
 - manageable complexity for small teams
-

4. BACKEND MODULE STRUCTURE

Core Modules

```
src/  
├─ auth/  
├─ users/  
├─ onboarding/  
├─ portfolio/  
├─ trading/  
├─ market/  
├─ watchlist/  
├─ ai/  
├─ websocket/  
├─ notifications/  
├─ admin/  
├─ prisma/  
├─ redis/  
├─ common/  
└─ config/
```

5. DATABASE ARCHITECTURE

Database

MySQL

ORM

Prisma ORM

Database Goals

- relational integrity
 - transaction consistency
 - scalable schema design
 - maintainable relationships
 - optimized querying
-

6. CORE DATABASE ENTITIES

Main Tables

User

Stores:

- account data
 - profile data
 - onboarding preferences
 - user level
 - settings
-

Portfolio

Stores:

- portfolio ownership
 - balance tracking
 - total value
 - performance data
-

Holding

Stores:

- owned assets

- quantities
 - average prices
 - P&L data
-

Transaction

Stores:

- buy orders
 - sell orders
 - timestamps
 - execution prices
 - transaction metadata
-

Watchlist

Stores:

- tracked assets
 - grouped watchlists
 - realtime subscriptions
-

AIConversation

Stores:

- AI interactions
 - contextual analysis
 - AI history
-

Notification

Stores:

- alerts
 - smart notifications
 - market intelligence updates
-

7. PRISMA ARCHITECTURE

Important Rules

Rule 1

Use Prisma migrations only.

Rule 2

Avoid unsafe raw SQL unless absolutely required.

Rule 3

Maintain relational consistency.

Rule 4

Always optimize relations carefully.

8. PRISMA FOLDER STRUCTURE

```
prisma/  
├─ schema.prisma  
├─ migrations/  
└─ seed/
```

9. DATABASE INDEXING STRATEGY

Important Index Areas

- stock symbols
- portfolio ownership
- transaction timestamps
- notification queries
- watchlist queries

- AI logs
-

10. DATABASE CONSISTENCY RULES

NEVER break:

- portfolio consistency
 - transaction history
 - user balances
 - AI context relationships
 - notification history
-

11. AUTHENTICATION ARCHITECTURE

Authentication System

JWT-based authentication.

Components

- access tokens
 - refresh tokens
 - HTTP-only cookies
 - auth guards
 - role guards
-

12. AUTH MODULE STRUCTURE

Main Components

```
auth/  
├─ controllers/  
├─ services/  
├─ strategies/  
├─ guards/  
├─ dto/  
└─ interfaces/
```

13. AUTHENTICATION FLOW

```
Login
↓
JWT Access Token
↓
Refresh Token
↓
Protected APIs
↓
Automatic Refresh
```

14. SECURITY RULES

Mandatory Rules

- backend-only secrets
- DTO validation
- route protection
- rate limiting
- secure cookies
- websocket validation

NEVER

- trust frontend validation
- expose API keys
- hardcode secrets
- bypass auth guards

15. ENVIRONMENT ARCHITECTURE

Important Principle

All secrets remain backend-only.

Required Variables

```
DATABASE_URL=  
JWT_SECRET=  
JWT_REFRESH_SECRET=  
REDIS_URL=  
OPENROUTER_API_KEY=  
TWELVEDATA_API_KEY=
```

16. MARKET DATA ARCHITECTURE

Primary Provider

TwelveData

Important Principle

Frontend NEVER directly communicates with providers.

All provider communication happens through backend abstraction.

17. PROVIDER ABSTRACTION SYSTEM

Architecture

```
Frontend  
↓  
Backend Market Service  
↓  
Provider Adapter Layer  
↓  
TwelveData
```

Benefits

- provider flexibility

- easier migration later
 - caching control
 - API optimization
-

18. REDIS ARCHITECTURE

Redis Responsibilities

- market caching
 - websocket coordination
 - AI response caching
 - session optimization
 - temporary realtime state
-

Important Rule

Use Redis aggressively for market-heavy systems.

19. WEBSOCKET ARCHITECTURE

Technology

Socket.IO

WebSocket Use Cases

- live stock updates
 - portfolio synchronization
 - smart alerts
 - watchlist updates
 - realtime notifications
-

20. WEBSOCKET FLOW

Market Provider

↓

```
Backend Market Service
↓
Redis Cache
↓
Socket.IO Gateway
↓
Frontend Subscribers
```

21. API ARCHITECTURE

API Style

REST-first architecture.

Future Support

Possible GraphQL layer later.

22. API VERSIONING

Required Structure

```
/api/v1
```

23. MAIN API MODULES

Core APIs

Auth APIs

- signup
 - login
 - refresh token
 - logout
-

Portfolio APIs

- holdings
 - performance
 - allocation
 - portfolio history
-

Trading APIs

- buy asset
 - sell asset
 - order history
 - simulated execution
-

Market APIs

- quotes
 - candles
 - market movers
 - sectors
 - stock search
-

Watchlist APIs

- create watchlist
 - add stock
 - remove stock
 - realtime subscriptions
-

AI APIs

- contextual analysis
 - AI assistant
 - news intelligence
 - portfolio insights
-

24. DTO VALIDATION STRATEGY

Mandatory Rule

Every API request MUST use DTO validation.

Validate

- request body
 - params
 - query strings
 - payload structures
-

25. ERROR HANDLING STRATEGY

Backend Errors Must

- remain structured
 - remain readable
 - avoid leaking internals
 - support frontend clarity
-

Standard Response Structure

```
{
  "success": false,
  "message": "Readable error message",
  "errorCode": "ERROR_CODE"
}
```

26. AI ARCHITECTURE

AI Name

YourBuy Intelligence

AI Provider

OpenRouter

AI Philosophy

AI MUST:

- remain contextual
 - understand viewed stocks
 - understand portfolio exposure
 - understand market context
 - understand news context
-

AI MUST NOT

- hallucinate financial data
 - promise profits
 - create fake certainty
 - behave generically
-

27. AI ORCHESTRATION ARCHITECTURE

Flow

```
Frontend Context
↓
Backend AI Service
↓
Prompt Orchestration
↓
```

```
OpenRouter
↓
AI Response Validation
↓
Frontend
```

28. AI CONTEXT ENGINE

AI context may include:

- viewed stocks
 - portfolio exposure
 - sector exposure
 - current page
 - market conditions
 - recent news
-

29. NEWS INTELLIGENCE FLOW

```
News
↓
AI Analysis
↓
Affected Sectors
↓
Impacted Stocks
↓
Educational Explanation
```

30. SMART ALERT ARCHITECTURE

Smart Alerts Include

- portfolio alerts
- stock alerts
- AI intelligence alerts
- news impact alerts
- sector movement alerts

31. BACKEND PERFORMANCE STRATEGY

Optimize

- Redis usage
- query efficiency
- websocket efficiency
- provider calls
- AI orchestration

32. MARKET API OPTIMIZATION STRATEGY

Important Principle

Protect API limits aggressively.

Use

- Redis caching
- batching
- subscription optimization
- stale-while-revalidate patterns

33. FILE STRUCTURE PHILOSOPHY

Codebase should remain:

- modular
 - predictable
 - scalable
 - readable
-

34. ENGINEERING RULES (VERY IMPORTANT)

Rule 1

Frontend NEVER directly calls providers.

Rule 2

Provider logic MUST remain abstracted.

Rule 3

WebSocket systems MUST remain optimized.

Rule 4

AI systems MUST remain contextual.

Rule 5

Redis MUST be used aggressively.

Rule 6

Backend modules MUST remain isolated.

Rule 7

All APIs MUST remain validated.

Rule 8

Security is NEVER optional.

Rule 9

Environment variables MUST remain centralized.

Rule 10

Always preserve scalability.

35. DEPLOYMENT ARCHITECTURE

Frontend

Vercel

Backend

Railway or Render

Database

MySQL

Cache

Redis

36. TESTING PHILOSOPHY

Backend testing should validate:

- authentication
- portfolio consistency
- websocket stability
- AI orchestration
- provider handling
- realtime updates

37. FINAL BACKEND SUMMARY

YourBuy's backend architecture is designed to provide:

- scalable fintech-style systems
- realistic trading workflows
- contextual AI orchestration
- optimized realtime infrastructure
- modular backend engineering
- maintainable long-term scalability

The backend prioritizes:

- correctness
- security
- scalability
- provider abstraction
- Redis optimization
- websocket stability
- AI intelligence consistency

while avoiding:

- backend chaos
- provider lock-in
- insecure implementations
- poor scalability
- unstructured APIs
- realtime inefficiency

END OF DATABASE_AND_API.md